Carmona Ayala Mariana Zoe

Silverio Martinez Andrés

Práctica final completa

1. **Configuración general del programa**

El programa permite seleccionar un canal analógico (AN0–AN7) desde la terminal. Cuando el usuario escribe un número del 0 al 7, el PIC selecciona ese canal, realiza una conversión analógica–digital de 8 bits y envía el resultado en formato decimal por UART a la terminal.

2. **UART**

La sección de UART se maneja usando directamente los registros del PIC16F877A:

- SPBRG: Determina la velocidad en baudios (9600 bps)
- TXSTA: Controla la transmisión
- RCSTA: Habilita la recepción
- TXREG: Almacena el byte a transmitir
- RCREG: Almacena el byte recibido
- TRMT: Indica si el transmisor está vacío
- RCIF: Indica si llegó un dato nuevo

Las funciones creadas son:

- recibe_dato(): Espera hasta que RCIF = 1, lo que significa que llegó un byte, y lo regresa desde RCREG.
- envia_dato(): Espera a que el transmisor esté libre (TRMT = 1) y envía un byte cargándolo en TXREG.
- envia_cadena(): Envía caracteres uno por uno hasta encontrar el fin de la cadena.
- envia_decimal(): Convierte un número de 0–255 en sus centenas, decenas y unidades usando aritmética básica, y envía cada dígito como ASCII.

3. **Configuración del ADC**

El ADC se configura usando:

- ADCON1: Define qué pines son analógicos y el formato del resultado
- TRISA / TRISE: Configuran los pines analógicos como entradas
- ADCON0: Selecciona el canal, activa el ADC y controla la conversión
- ADRESH: Contiene los 8 bits más significativos del resultado (MSB)

Al usar ADCON1 = 0b00000000, habilitamos todos los canales analógicos (AN0-AN7) y selecciona:

- Justificación izquierda (ADFM = 0)
- Vref+ = VDD
- Vref− = GND

Con ADFM = 0, los 8 bits más importantes quedan en ADRESH, lo cual cumple la restricción de usar solo 8 bits.

4. **Programa principal**

El programa inicia configurando la comunicación serial del PIC16F877A para trabajar a 9600 baudios, habilitando tanto el transmisor como el receptor sin utilizar librerías externas. Esta configuración permite que el microcontrolador envíe mensajes a la terminal y reciba el canal analógico que el usuario desea convertir. Después de configurar la UART, el programa establece también el módulo ADC: habilita todos los canales analógicos (AN0–AN7), selecciona justificación a la izquierda y define las referencias de voltaje internas (VDD y VSS). Con esto, el sistema queda listo para recibir comandos y realizar conversiones analógicas-digitales.

Una vez que la configuración inicial está completa, el programa muestra un mensaje de bienvenida en la terminal y solicita al usuario que escriba un número entre 0 y 7 para seleccionar el canal analógico. Cuando el usuario envía un carácter por UART, el microcontrolador lo recibe mediante la función recibe_dato(). El programa convierte este carácter ASCII en un valor numérico real, lo que permite identificar correctamente el canal ANx que se desea medir.

Posteriormente, el canal seleccionado se carga en los bits CHS2:CHS0 del registro ADCON0, lo que conmuta físicamente el multiplexor analógico hacia el canal indicado. Antes de iniciar la conversión, el programa espera un tiempo de adquisición adecuado para que el capacitor interno del ADC se cargue al nivel del voltaje presente en el pin correspondiente. Una vez transcurrido este tiempo, el programa comienza la conversión asignando un 1 al bit GO/DONE, e inmediatamente entra en un ciclo de espera hasta que la conversión finaliza automáticamente.

Cuando el bit GO/DONE regresa a cero, el ADC ha finalizado su proceso y el resultado ya está disponible. Debido a la justificación a la izquierda configurada previamente, los ocho bits más significativos de la conversión se encuentran directamente en el registro ADRESH, lo que permite trabajar únicamente con esos 8 bits como lo requiere la práctica. El programa toma ese valor, lo convierte al formato decimal usando la función envia_decimal() y lo envía por UART a la terminal, donde

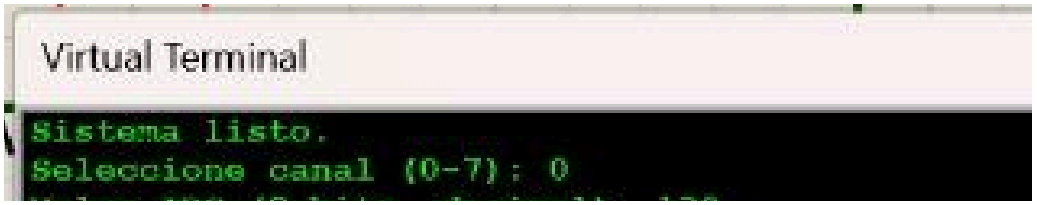
el usuario puede visualizar el número correspondiente al nivel analógico del canal seleccionado.

Finalmente, el proceso se repite indefinidamente. Después de imprimir el valor convertido, el programa vuelve a solicitar un canal y queda en espera de un nuevo número por parte del usuario. Gracias a este ciclo, es posible medir distintos canales del ADC en tiempo real según la entrada del usuario, cumpliendo así con los objetivos de la práctica.

5. Ejecución de la práctica y resultados obtenidos

Una vez configurado el circuito en Proteus con el PIC16F877A, el módulo ADC y la comunicación Serial mediante la terminal virtual, se cargó el archivo .hex generado en mikroC y se inició la simulación.

El sistema mostró inmediatamente el mensaje:



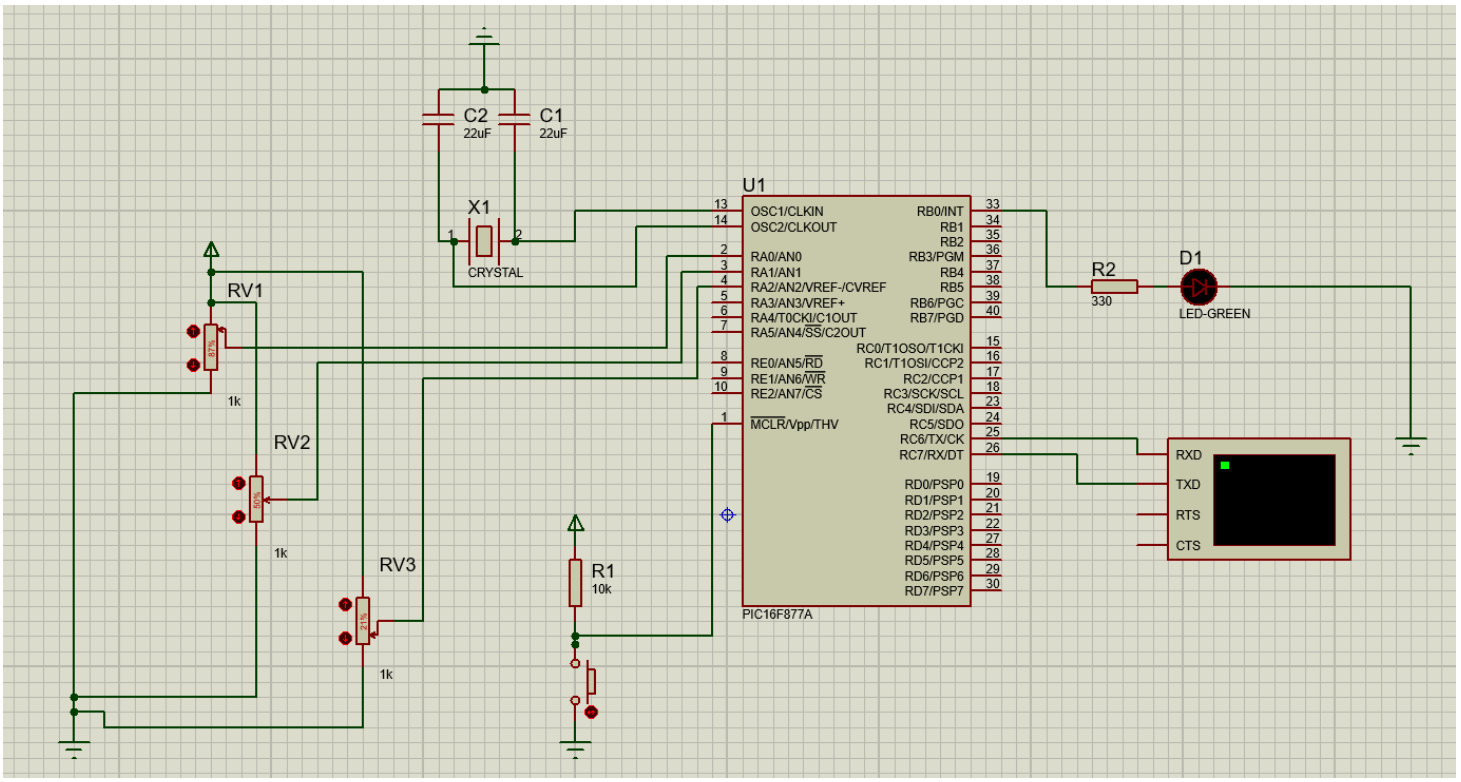
Esto confirma que la UART está configurada correctamente a 9600 baudios y que el PIC está ejecutando el programa desde el inicio del main().

Para realizar las pruebas, se conectaron potenciómetros a los canales analógicos AN0, AN1 y AN2, y se escribieron en la terminal los números correspondientes al canal que se deseaba convertir. Cada vez que se seleccionaba un canal, el PIC realizaba:

1. Lectura del caracter por UART
2. Configuración del canal ADC según el número recibido
3. Conversión analógica-digital de 8 bits
4. Envío del resultado en formato decimal a la terminal

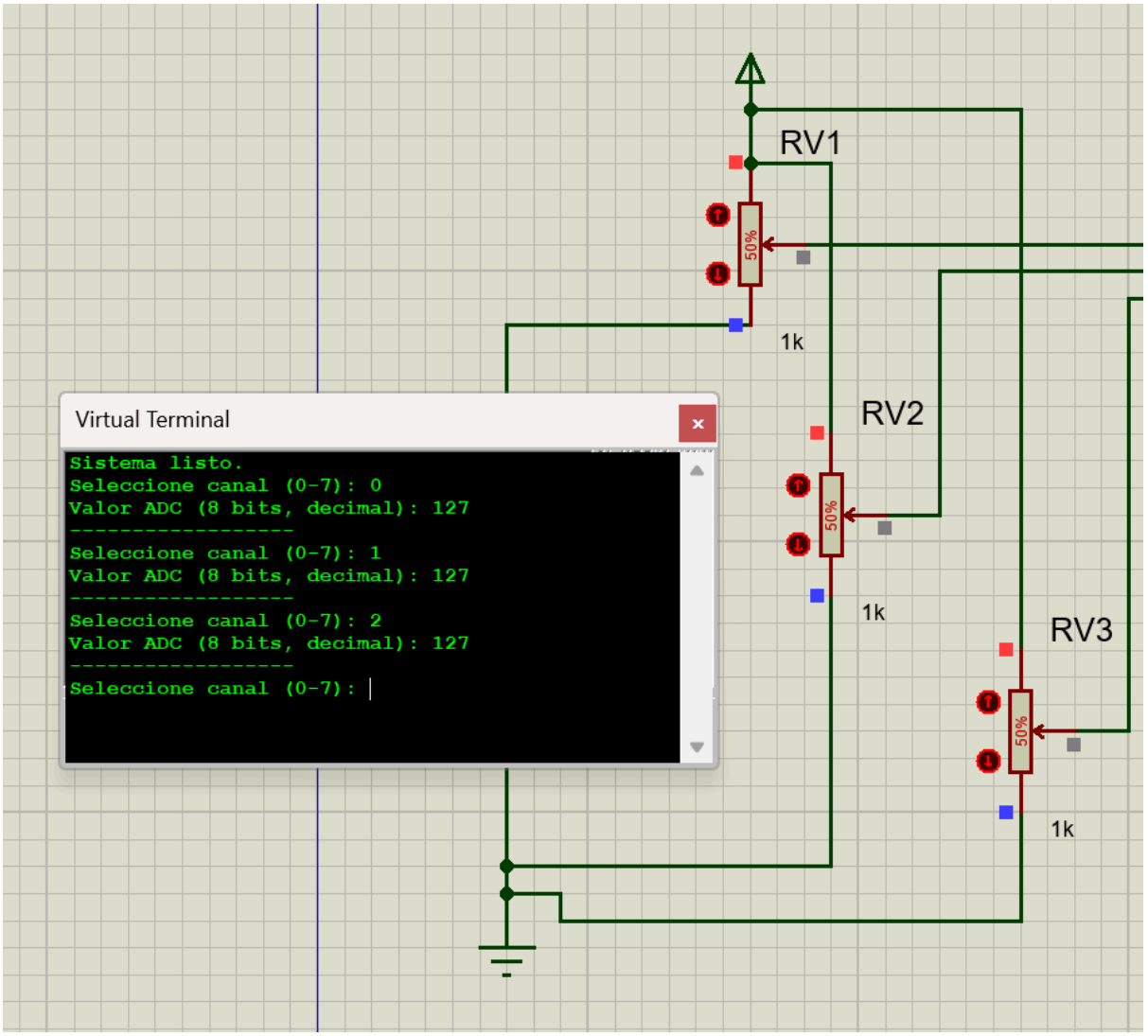
En la siguientes figuras se muestran las pruebas realizadas para diferentes canales y distintas posiciones de los potenciómetros:

Primero, se muestra nuestro circuito armado, el cual tomamos como base el que la profesora utilizó en clase, y lo modificamos de acuerdo a nuestras necesidades, quedando de la siguiente manera:



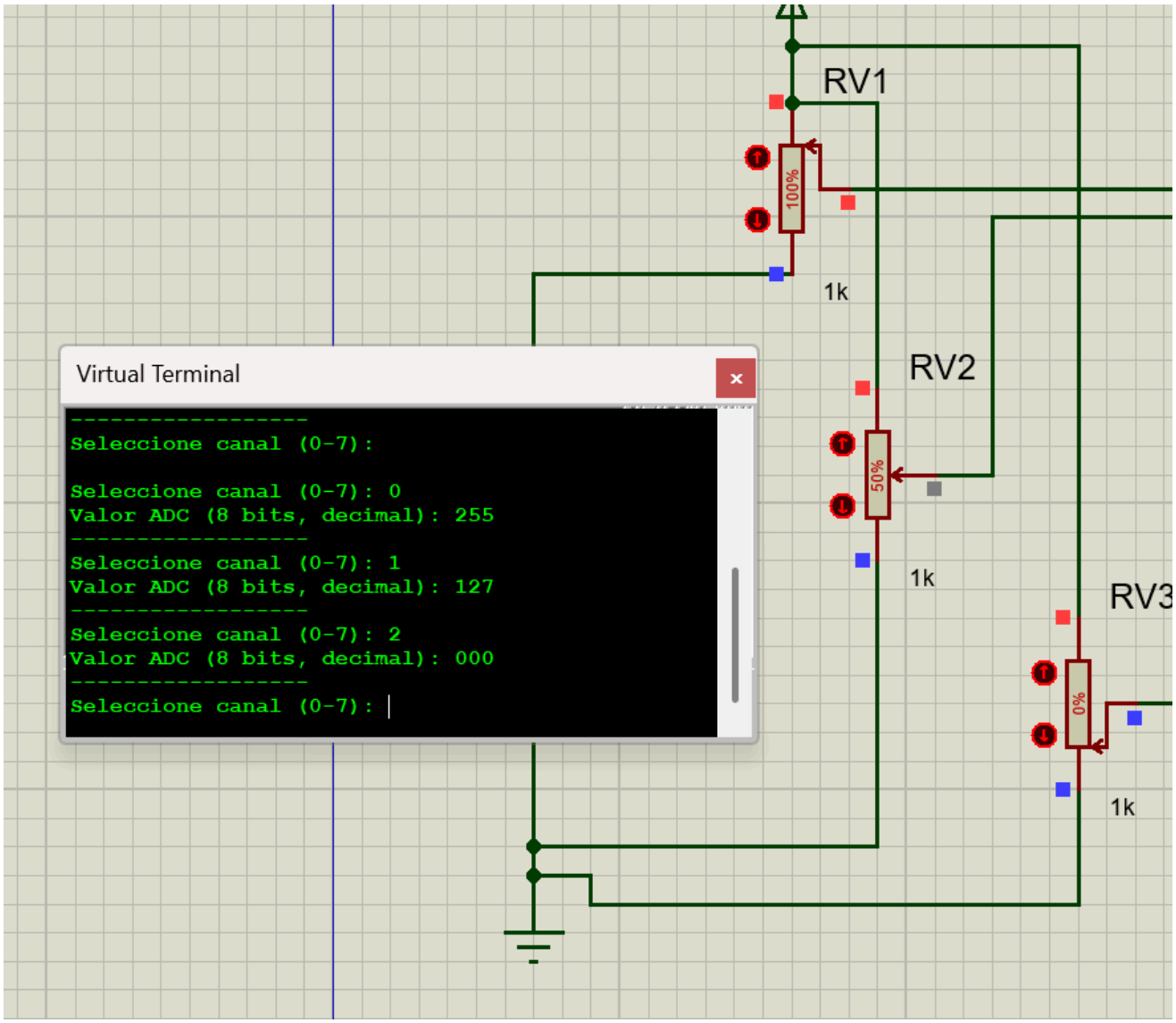
Para las pruebas, solo conectamos en los primeros 3 conectores analógicos (RA0/AN0, RA1/AN1 y RA2/AN2).

Ahora bien, ejecutando desde proteus, y seleccionando cada uno de los canales, da los siguientes resultados:



Donde, el valor resultante de cada uno de los canales es, el valor del voltaje real que recibe la PIC16F877A en tiempo real. Como los 3 potenciómetros se encuentran en el 50%, da el valor 127, que es la mitad de 255, y lo cual es la interpretación digital del voltaje que entra al pin del PIC.

Otra prueba que se realizó es la siguiente, con los valores de los potenciómetros variando:



Donde, se nota mejor como el donde se nota mejor como el sistema realiza la conversión ADC de forma precisa y lineal, asignando el valor correcto de 8 bits (0-255) según el porcentaje de voltaje en cada canal seleccionado.

6. Conclusiones

- Carmona Ayala Mariana Zoe

Este último ejercicio me ayudó a consolidar varios conceptos que vimos durante el curso, especialmente el manejo del ADC y de la comunicación UART. Fue necesario entender cómo funcionan los registros del PIC16F877A y qué hace cada bit para que el programa pueda seleccionar canales desde la terminal y realizar la conversión correctamente.

Algo que sí nos tomó tiempo fueron las complicaciones con Proteus, principalmente con la terminal virtual y la alimentación del microcontrolador, ya que pequeños detalles en la simulación provocaban que el ADC no funcionara o que aparecieran advertencias de canales mal configurados. Resolver estos problemas nos permitió comprender mejor cómo deben conectarse los periféricos y la importancia de seguir las recomendaciones del datasheet. En general, este ejercicio cerró bien el laboratorio porque combinó programación y simulación.

- Silverio Martínez Andrés

Para mí este ejercicio final fue una buena oportunidad para aplicar de manera práctica todo lo que aprendimos durante el curso, principalmente en las últimas sesiones. Trabajar directamente con los registros del ADC y de la UART me ayudó a entender más a fondo cómo funciona el hardware y qué sucede detrás de las funciones que normalmente se usan en un entorno más automatizado.

Durante la realización del ejercicio tuvimos varios problemas con Proteus, sobre todo con la terminal virtual y la configuración de los pines analógicos. Estas dificultades nos obligaron a revisar cada conexión, validar la alimentación del PIC y corregir detalles que normalmente pasarían desapercibidos. Aunque fue frustrante al inicio, al final sirvió para reforzar nuestra habilidad de depurar tanto el circuito como el código. Me voy satisfecho porque el programa terminó funcionando como se pedía y porque comprendí mejor el proceso completo de conversión A/D y comunicación serial.